程序设计原理

C++字符串

任课教师： 喻　梅
于瑞国
王建荣
李雪威
赵满坤

Why do we need strings?

Motivation

In our daily programs, we often need to deal with **textual information** — names, addresses, messages, book titles, or any other words and sentences.

Numbers (*int, double*) cannot represent text,

so we need a **special data type** to store and process a sequence of characters.

That data type is ***string***.

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string name = "Alice";
    cout << "Hello, " << name << "!" << endl;
    return 0;
}
```

Declaring and Initializing Strings

Strings can be initialized (assigned a value) in several ways:

```
string s1 = "Hello";       // direct initialization
string s2("World");        // constructor style
string s3;                 // empty string
s3 = "C++";                // assignment after declaration
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string greeting = "Welcome to C++!";
    cout << greeting << endl;
    return 0;
}
```

Reading and Printing Strings

Use `cin >>` to read an item into a string:

```
string firstName;
cin >> firstName;
```

Use `getline` function to put a line of input, possibly including spaces, into a string:

```
string sentence;
getline(cin, sentence);
```

Reading Strings: cin vs getline()

cin >> reads a single word only.It stops reading when a space, tab, or newline is encountered.

```
string name;
cout << "Enter your name: ";
cin >> name;
cout << "Hi, " << name << "!";
```

Input：
Alice Johnson
Output:
Hi, Alice!

getline() reads an entire line, including spaces:

```
string address;
cout << "Enter your address: ";
getline(cin, address);
cout << "Address: " << address;
```

Input：
123 Nanjing Road,Tianjin
Output:
Address: 123 Nanjing Road, Tianjin

Reading Strings: cin vs getline()

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    int age;
    string name;

    cout << "Enter your age: ";
    cin >> age;              // reads a number

    cout << "Enter your name: ";
    getline(cin, name);      // tries to read a full line

    cout << "Age: " << age << ", Name: " << name << endl;
    return 0;
}
```

Input：
18⏎
Alice Johnson⏎

Output:
Age: 18, Name:

Reading Strings: cin vs getline()

After cin >> age, the **newline character ('\n')** from pressing **Enter** is still **waiting in the input buffer**:

```
[ '1' '8' '\n' A l i c e ... ]
          ↑
   getline() starts reading here
```

```
cout << "Enter your age: ";
cin >> age;
cin.ignore();       //discard the leftover newline

cout << "Enter your name: ";
getline(cin, name);
```

Output：
Age: 18, Name: Alice Johnson

Common String Operations

操作	示例代码	说明
拼接	`s3 = s1 + s2;`	合并两个字符串
追加	`s1 += "!";`	在末尾追加内容
长度	`s1.length() / s1.size()`	返回字符数
清空	`s1.clear()`	变为空字符串
判断空	`s1.empty()`	返回true或false

```
string s = "Hi";
s += " there!";
cout << s << " (" << s.length() << " chars)";
```

Hi there! (9 chars)

string Comparison

C++ allows you to compare string objects directly using the same relational operators as numbers：

== != < <= > >=

```
string a = "apple";
string b = "banana";

if (a < b)
    cout << a << " comes before " << b << endl;
```

apple comes before banana

string Comparison

You can also use the string.compare() member function.

```
string s1 = "Hello";
string s2 = "Hi";

int result = s1.compare(s2);
```

Result	Meaning
0	strings are equal
< 0	s1 is less than s2
> 0	s1 is greater than s2

Substring and Searching

Getting a Substring — substr()： The substr() function extracts part of a string.

Syntax:

```
// 形式1：只指定起始位置，截取从该位置到字符串末尾的子串
string substr(size_t pos) const;
// 形式2：指定起始位置和子串长度，截取固定长度的子串
string substr(size_t pos, size_t count) const;
```

```
string s = "programming";

cout << s.substr(0, 3) << endl;   // "pro"
cout << s.substr(3, 4) << endl;   // "gram"
cout << s.substr(7) << endl;      // "ming"
```

Substring and Searching

The find() function searches for a substring or character and returns the index of the first match.

Syntax:

size_t find(const string& str, size_t pos = 0) const;

```
string s = "Welcome to C++ programming";

cout << s.find("come") << endl;   // 3
cout << s.find('o') << endl;      // 4
cout << s.find("Java") << endl;   // string::npos (not found)
cout << s.find("o", 5)<< endl;   // search from index 5 onward
```

```
string email = "alice@tju.edu.cn";
int pos = email.find('@');
string user = email.substr(0, pos);
cout << "Username: " << user;d
```

Username: alice

Substring and Searching

The find() function searches for a substring or character and returns the index of the first match.

Syntax:

```
// 形式1：只指定起始位置，截取从该位置到字符串末尾的子串
string substr(size_t pos) const;
// 形式2：指定起始位置和子串长度，截取固定长度的子串
string substr(size_t pos, size_t count) const;
```

```
string s = "programming";

cout << s.substr(0, 3) << endl;   // "pro"
cout << s.substr(3, 4) << endl;   // "gram"
cout << s.substr(7) << endl;      // "ming"
```

String Operators

Operator	Description
[]	Accesses characters using the array subscript operators.
=	Copies the contents of one string to the other.
+	Concatenates two strings into a new string.
+=	Appends the contents of one string to the other.
<<	Inserts a string to a stream
>>	Extracts characters from a stream to a string delimited by a whitespace or the null terminator character.
==, !=, <, <=, >, >=	Six relational operators for comparing strings.

string	
+string()	Constructs an empty string.
+string(value: string)	Constructs a string with the specified string literal value.
+string(value: char[])	Constructs a string with the specified character array.
+string(ch: char, n: int)	Constructs a string initialized with the specified character n times.
+append(s: string): string	Appends string s into this string object.
+append(s: string, index: int, n: int): string	Appends n number of characters in s starting at the position index to this string.
+append(s: char[], n: int): string	Appends the first n number of characters in s to this string.
+append(n: int, ch: char): string	Appends n copies of character ch to this string.
+assign(s: char[]): string	Assigns array of characters or a string s to this string.
+assign(s: string, index: int, n: int): string	Assigns n number of characters in s starting at the position index to this string.
+assign(s: string, n: int): string	Assigns the first n number of characters in s to this string.
+assign(n: int, ch: char): string	Assigns n copies of character ch to this string.
+at(index: int): char	Returns the character at the position index from this string.
+length(): int	Returns the number of characters in this string.
+size(): int	Same as length().
+capacity(): int	Returns the size of the storage allocated for this string.
+clear(): void	Removes all characters in this string.
+erase(index: int, n: int): string	Removes n characters from this string starting at position index.
+empty(): bool	Returns true if this string is empty.
+compare(s: string): int	This two compare functions are like the strcmp function in §7.9.4, “String Functions,” with the same return value.
+compare(index: int, n: int, s: string): int	
+copy(s: char[], index: int, n: int): void	Copies n characters into s starting at position index.
+data(): char[]	Returns a character array from this string.
+substr(index: int, n: int): string	Returns a substring of n characters from this starting at position index.
+substr(index: int): string	Returns a substring of this string starting at position index.
+swap(s: string): void	Swaps this string with s.
+find(ch: char): int	Returns the position of the first matching character for ch.
+find(ch: char, index: int): int	Returns the position of the first matching character for ch at or from the position index.
+find(s: string): int	Returns the position of the first matching substring s.
+find(s: string, index: int): int	Returns the position of the first matching substring s starting at or from the position index.
+replace(index: int, n: int, s: string): string	Replaces the n characters starting at position index in this string with the string s.
+insert(index: int, s: string): string	Inserts the string s into this string at position index.
+insert(index: int, n: int, ch: char): string	Inserts the character ch n times into this string at position index.

Exercise

Counting Characters

Write a program which counts and reports the number of each alphabetical letter. Ignore the case of characters.

Input

A sentence in English is given in several lines.

Output

Prints the number of alphabetical letters in the following format:

a : The number of 'a'
b : The number of 'b'
c : The number of 'c'
.
.
z : The number of 'z'

Constraints

The number of characters in the sentence < 1200

Sample Input

This is a pen.

Sample Output

a : 1	j : 0	s : 2
b : 0	k : 0	t : 1
c : 0	l : 0	u : 0
d : 0	m : 0	v : 0
e : 1	n : 1	w : 0
f : 0	o : 0	x : 0
g : 0	p : 1	y : 0
h : 1	q : 0	z : 0
i : 2	r : 0	

Exercise

Toggling Cases

Write a program which converts uppercase/lowercase letters to lowercase/uppercase for a given string.

Input

A string is given in a line.

Output

Print the converted string in a line. Note that you do not need to convert any characters other than alphabetical letters.

Constraints

The length of the input string < 1200

Sample Input

fAIR, LATER, OCCASIONALLY CLOUDY.

Sample Output

Fair, later, occasionally cloudy.

Exercise

Card Game

Taro and Hanako are playing card games. They have n cards each, and they compete n turns. At each turn Taro and Hanako respectively puts out a card. The name of the animal consisting of alphabetical letters is written on each card, and the bigger one in lexicographical order becomes the winner of that turn. The winner obtains 3 points. In the case of a draw, they obtain 1 point each.

Write a program which reads a sequence of cards Taro and Hanako have and reports the final scores of the game.

Input

In the first line, the number of cards n is given. In the following n lines, the cards for n turns are given respectively. For each line, the first string represents the Taro's card and the second one represents Hanako's card.

Exercise

Constraints

n ≤ 1000
The length of the string ≤ 100

Output

Print the final scores of Taro and Hanako respectively. Put a single space character between them.

Sample Input

3
cat dog
fish fish
lion tiger

Sample Output

1 7

Exercise

Ring

Write a program which finds a pattern p in a ring shaped text s.

Input

In the first line, the text s is given.
In the second line, the pattern pis given.

Output

If p is in s, print Yes in a line, otherwise No.

Constraints

1≤length of p≤ length of s≤100
s and p consists of lower-case letters

Sample Input 1

vanceknowledgetoad
advance

Sample Output 1

Yes

Sample Input 2

vanceknowledgetoad
advanced

Sample Output 2

No

Exercise

Finding a Word

Write a program which reads a word W and a text T, and prints the number of word W which appears in text T. T consists of string Ti separated by space characters and newlines. Count the number of Ti which equals to W. The word and text are case insensitive.

Constraints

The length of W ≤ 10

W consists of lower case letters

The length of T in a line ≤ 1000

Input

In the first line, the word W is given. In the following lines, the text T is given separated by space characters and newlines.

"END_OF_TEXT" indicates the end of the text.

Output

Print the number of W in the text.

Sample Input

computer

Nurtures computer scientists and highly-skilled computer engineers

who will create and exploit "knowledge" for the new era.

Provides an outstanding computer environment.

END_OF_TEXT

Sample Output

3